Final Project

Interactive Graphics

2025-26

Luca Conti, 1702084

Samuele Tomassacci, 2216267

Contents

1 Introduction

1.1 Project Overview

Night at the Museum is an interactive 3D experience inside a museum inspired by the movie of the same name. The project was built with Three.js and JavaScript.
It integrates a navigable 3D environment, hierarchical and skeletal models, procedural and tween-based animations, multiple camera modes, collision detection, raycast-based interaction, image-based textures, procedurally generated materials, configurable lighting, shadows, fog, and an HTML-based user interface.
The player controls a security guard who can freely move through a central atrium and four themed exhibition rooms dedicated to dinosaurs, statues, ancient Egypt, and paintings. During the exploration, the player can open sliding doors, inspect exhibits, change the displayed paintings, awaken an animated T-Rex, interact with a Moai statue, and open a sarcophagus containing a mummy. A global awakening event transforms the museum by opening its doors, changing the lighting, and bringing some of its exhibits to life.

1.2 Requirements

The project was developed in accordance with the following requirements:

- The scene includes at least one hierarchical model.
- Different types of lighting and texture maps are used, including colour, normal, and roughness maps.
- User interaction is supported through both mouse and keyboard input.
- Most scene objects are animated. All animations were implemented directly in JavaScript using Tween.js and trigonometric functions.
- The application runs directly in a web browser and does not require additional software installation, compilation, or an application server.

2 Environment, Libraries and Project Organization

The project runs as a static browser application built with Vite. Rendering is performed with WebGL through Three.js. The WebGL canvas is created by `THREE.WebGLRenderer` and added to the page at runtime, while the HTML page provides the UI root and the CSS styles for the overlay controls.

2.1 Environment

- **HTML**: defines the page shell, viewport metadata and the `ui-root` container used by the interface.
- **CSS**: styles the full-screen canvas and the museum control panel, including camera, lighting, shadows and quality controls.
- **JavaScript ES modules**: implement scene creation, model loading, materials, lights, animation, input, collision and interaction systems.
- **Vite**: provides the development server and bundles the application for production.

2.2 Libraries and addons not developed by the authors

Library or addon	Use in the project
`three`	Main 3D library used for WebGL rendering, scene graph management, cameras, lights, shadows, geometry, materials, textures and math utilities.
`GLTFLoader`	Three.js addon used to load the local `.glb` models, including the player, T-Rex, mummy, coffin, Moai statue and museum props.
`@tweenjs/tween.js`	Used for timed animations such as door sliding, sarcophagus opening, Moai mouth movement, statue hopping and player walk blending.
Browser WebGL implementation	GPU-backed rendering target used internally by `THREE.WebGLRenderer`.
Browser Canvas 2D API	Used to generate procedural textures at runtime, later converted into `THREE.CanvasTexture`.

Table 1: External libraries and browser APIs used in the project.

The main files of the project are shown in Table 2.

3 3D Models

The project includes several external 3D models that were not created by us. The imported `.glb` models, described in Table 3, are stored in the `src/models` directory and loaded at runtime with `GLTFLoader`. These assets include the policeman character, the T-Rex skeleton, the Moai statue, the sarcophagus, the mummy, benches, ceiling lights and Ionic columns. They provide the base meshes, materials and, in some cases, skeletal structures used in the scene: the security guard character and the T-Rex are indeed complex hierarchical models.

File	Use
index.html	Declares the application container and loads the JavaScript entry point through Vite.
main.js	Entry point: creates the scene, renderer, camera, player, museum, lighting system, collision system, interaction system, UI controller and main animation loop.
core/Scene.js, core/Renderer.js, core/Camera.js	Scene configuration, WebGL renderer settings, camera modes, camera orbiting, first-person/third-person views, top-down view, panorama mode and camera collision handling.
core/factory.js	Shared object factory used to build repeated museum elements such as walls, floors, sliding doors, signs, benches, columns, chandeliers and wall sconces. It also loads reusable GLB props.
world/Museum.js, world/Hall.js, world/GalleryRoom.js, world/EgyptianRoom.js, world/DinosaurRoom.js, world/StatueRoom.js	World modules to assemble the museum layout, central hall, themed rooms, doors, static props and interactive exhibits and expose them.
entities/Player.js	Loads and controls the playable policeman character.
entities/Dinosaur.js, entities/Statue.js, entities/Sarcophagus.js, entities/Paintings.js	Interactive exhibit entities.
systems/Lighting.js, systems/Collision.js, systems/Interaction.js, systems/DoorInteraction.js, systems/Animation.js, systems/UIController.js, systems/ExhibitPrompt.js	Gameplay and support systems: lighting modes, flashlight control, collision constraints, raycast-based clicking, sliding door interaction, per-frame update dispatch, HTML UI controls and contextual exhibit prompts.
utils/materials.js, utils/proceduralTextures.js, utils/math.js	Utility modules for shared materials, generated canvas textures and reusable math helpers such as damping and 2D distance checks.
src/models/	Imported GLB assets and reusable prop models.
src/texture/	Texture assets.

Table 2: Project Files.

All animation behaviour is implemented manually in JavaScript and discussed in the following sections.

Model	Technical use
Policeman	Player character; 103 nodes, 11 meshes, one skin, and 66 joints.
T-Rex	Main complex exhibit; 121 nodes, two meshes, one skin, and 100 joints.
Moai statue	Statue exhibit and custom mouth deformation; eight nodes and one small skin.
Coffin	Sarcophagus body and separately moved lid components.
Mummy	Static model revealed inside the opened coffin.
Modern bench	Reused furniture model in the atrium and exhibition rooms.
Ceiling light	Reused decorative fixture in each room.
Ionic column	Reused architectural decoration.

Table 3: Imported 3D models used in the project.

4 Animations

Since the assignment prohibits imported animation, all animations used by the application are implemented in JavaScript. The animation system combines per-frame updates, direct bone transformations, procedural mathematical functions, and `tween.js`. The render loop uses `requestAnimationFrame` and updates the animated objects using elapsed time and frame delta time. Tweening and easing functions are provided by `@tweenjs/tween.js`, while more complex movements are calculated procedurally by the application.

4.1 T-Rex hierarchy and animation

The T-Rex is the most complex hierarchical model in the project. Its skeleton includes separate bones for the body, legs, feet, arms, neck, head, jaw, and tail. These components are controlled individually to create walking, tail movement, body oscillation, leg compensation, head movement, and roaring animations. The T-Rex joint hierarchy used during the animations is reported in Table 4.

The tail is animated as a chain of six bones, ordered from the base of the tail to its tip. The amplitudes of the swing increase progressively toward the end of the tail. Each segment receives a sinusoidal quaternion rotation with a slightly different phase. This delay produces a traveling wave instead of making the whole tail rotate as a rigid object.

The walking animation alternates the left and right foot-control joints. Each step lifts one foot forward and then lowers it into the next contact position, using cosine easing to smooth the

Animation branch	**Used joint hierarchy**
Animated body root	Mover_ArmatureRexy
Tail	Mover_ArmatureRexy ↪ Cola1_ArmatureRexy ↪ Cola2_ArmatureRexy ↪ Cola3_ArmatureRexy ↪ Cola4_ArmatureRexy ↪ Cola5_ArmatureRexy ↪ Cola6_ArmatureRexy
Chest, neck, head, and jaw	Mover_ArmatureRexy ↪ Pecho001_ArmatureRexy ↪ Cuello_ArmatureRexy ↪ Cuello001_ArmatureRexy ↪ Atlas_ArmatureRexy ↪ Atlas001_ArmatureRexy ↪ Cabeza_ArmatureRexy ↪ Boca_ArmatureRexy ↪ CabezaIK_ArmatureRexy
Left arm	Mover_ArmatureRexy ↪ AntebrazoL_ArmatureRexy ↪ BrazoL_ArmatureRexy
Right arm	Mover_ArmatureRexy ↪ AntebrazoR_ArmatureRexy ↪ BrazoR_ArmatureRexy
Left deforming leg	Mover_ArmatureRexy ↪ MusloL_ArmatureRexy ↪ PiernaL_ArmatureRexy
Right deforming leg	Mover_ArmatureRexy ↪ MusloR_ArmatureRexy ↪ PiernaR_ArmatureRexy
Left IK target	PiernaIKL_ArmatureRexy ↪ TalonL_ArmatureRexy
Right IK target	PiernaIKR_ArmatureRexy ↪ TalonR_ArmatureRexy

Table 4: T-Rex joint hierarchy used during walking, tail, and roar animations.

start and end of the motion. The T-Rex descends gradually from the platform during the first walking steps. The walking animation can execute a fixed number of steps, as in the museum-awakening sequence, or continue until it is explicitly stopped.
The roar is a coordinated full-body animation that combines a step, jaw opening, neck movement, head rotation, body displacement, arm rotation, and vibration. The sequence begins with a step of the left foot. During this phase, the jaw starts opening and, after the step, it continues to its maximum opening angle. As the jaw opens, the bones of the neck and head are progressively rotated, the body leans forward, advances slightly, and lowers during the step, while the arm bones rotate. Once the jaw is fully open, a small high-frequency quaternion rotation is applied to the body using sinusoidal function, and the pose is maintained for 2 s. to simulate the physical intensity of the roar. Finally, the jaw closes using cosine easing. The neck, head, body, arms, and step gradually return toward their original transforms at the same time.

4.2 Player hierarchy and animation

The player is controlled through a root `THREE.Group`. Its animation system directly controls the hierarchy of the character and modifies the spine, arms, forearms, hands, fingers, legs, feet, and toes to produce a procedural walking cycle. The character joint hierarchy used during the animations is reported in Table 5.
The walk cycle is represented by a phase, ranging over 0 to 2π, and a strength value that controls how visible the motion is. Tween.js advances the phase linearly over a 760 ms cycle and repeats it while movement input is active. When movement starts or stops, short easing tweens fade the strength in or out, so the character does not snap abruptly between idle and walking. Sinusoidal functions derive opposite limb phases, a vertical body bounce, foot lift, and trailing-foot behaviour.
The right arm receives a special pose to hold the flashlight. The wrist is bent and the fingers are curled with different angles for the thumb, index, middle, ring, and little finger. The flashlight follows the transformations of the right arm hierarchy.

4.3 Moai deformation and movement

The Moai statue animation is implemented by combining a custom shader based on mouth deformation with interpolation provided by `tween.js`. After loading the GLB model, the code analyses its vertices in the local coordinate system of the face and identifies the region corresponding to the mouth. For each vertex in this region, two custom buffer attributes

Animation branch	**Used joint hierarchy**
Trunk	mixamorig:Spine_02
Left arm	mixamorig:LeftArm_09 ↪ mixamorig:LeftForeArm_010 ↪ mixamorig:LeftHand_011
Right arm and grip	mixamorig:RightArm_033 ↪ mixamorig:RightForeArm_034 ↪ mixamorig:RightHand_035 ↪ mixamorig:RightHandThumb1_036 ↪ mixamorig:RightHandThumb2_037 ↪ mixamorig:RightHandThumb3_038 ↪ mixamorig:RightHandIndex1_040 ↪ mixamorig:RightHandIndex2_041 ↪ mixamorig:RightHandIndex3_042 ↪ mixamorig:RightHandMiddle1_044 ↪ mixamorig:RightHandMiddle2_045 ↪ mixamorig:RightHandMiddle3_046 ↪ mixamorig:RightHandRing1_048 ↪ mixamorig:RightHandRing2_049 ↪ mixamorig:RightHandRing3_050 ↪ mixamorig:RightHandPinky1_051 ↪ mixamorig:RightHandPinky2_052 ↪ mixamorig:RightHandPinky3_053
Left leg	mixamorig:LeftUpLeg_055 ↪ mixamorig:LeftLeg_056 ↪ mixamorig:LeftFoot_057 ↪ mixamorig:LeftToeBase_058
Right leg	mixamorig:RightUpLeg_060 ↪ mixamorig:RightLeg_061 ↪ mixamorig:RightFoot_062 ↪ mixamorig:RightToeBase_063

Table 5: Policeman joint hierarchy used by the procedural animations.

are generated: `aMouthOffset`, which stores the displacement that the vertex should follow when the mouth opens, and `aMouthDarkness`, which stores how much the fragment should be darkened to simulate the inside of the mouth. Instead of replacing the whole material, it has been used the `onBeforeCompile` to modify the vertex and fragment shaders of the original `MeshStandardMaterial`; this preserves Three.js lighting, shadows and material response while adding the procedural mouth effect. The main uniform is `uMouthOpen`, a scalar parameter in the range $[0, 1]$ that controls the amount of deformation. In the vertex shader, each mouth vertex is displaced by `aMouthOffset * uMouthOpen`; in the fragment shader, the color is mixed with a darker version of itself according to `aMouthDarkness * uMouthOpen`. Therefore, as the mouth opens, the geometry moves and the interior becomes visually darker.

The temporal evolution of `uMouthOpen` is controlled with `tween.js`. When the player clicks the statue, a sequence of syllables is played: each syllable defines an opening value, an opening duration, a closing duration and a pause. Two chained tweens interpolate `mouthState.open` from zero to the target opening value and then back to zero using a sinusoidal easing function. On every tween update, the value is copied into the shader uniform, producing a smooth talking motion.

Listing 1: Modified vertex shader code for the Moai mouth animation.

```
shader.vertexShader = shader.vertexShader
          .replace(
            "#include <common>",
            `
            #include <common>
            attribute vec3 aMouthOffset;
            attribute float aMouthDarkness;
            uniform float uMouthOpen;
            varying float vMouthDarkness;
            `
          )
          .replace(
            "#include <begin_vertex>",
            `
            vec3 transformed = vec3(position);
            transformed += aMouthOffset * uMouthOpen;
            vMouthDarkness = aMouthDarkness * uMouthOpen;
            `
          );
```

Listing 2: Modified fragment shader code for darkening the inside of the mouth.

```
shader.fragmentShader = shader.fragmentShader
          .replace(
            "#include <common>",
            `
            #include <common>
            varying float vMouthDarkness;
            `
          )
          .replace(
            "#include <dithering_fragment>",
            `
            float mouthShade = clamp(vMouthDarkness, 0.0, 1.0);
            diffuseColor.rgb = mix(
              diffuseColor.rgb,
              diffuseColor.rgb * ${MOUTH_INTERIOR_DARKNESS.toFixed(2)},
              mouthShade
            );
            #include <dithering_fragment>
            `
          );
```

The statue's body movement is separate from the mouth deformation. Its `modelRoot` hops through waypoint lists using a 620 ms quadratic tween. A small tilt is added during each jump.

4.4 Other animated objects

Clicking the sarcophagus produces the opening of the exhibit, holds the open state, and then closes it automatically. The mummy remains inside the coffin and is revealed by the moving lid.

Each museum door stores closed and open panel positions in its group metadata. When activated, two quadratic tweens move the panels in opposite directions over 750 ms. A door normally opened by pressing `O` closes after 3.5 seconds, while the doors opened by the global awakening remain open.

The nearest painting becomes emissive and oscillates. The oscillation target is sinusoidal, and the actual angle approaches it smoothly using the frame delta.

5 Texture and Material Information

The project uses several external image textures stored in the folder `src/texture`. Some textures are not loaded from image files. The project also generates procedural textures directly in JavaScript using the Canvas 2D API. These are used, for example, for wood-like decorative materials and Egyptian-style decorations. In this case, the texture is drawn at runtime and then converted into a Three.js texture. Further details are reported in Table 6.

These textures are combined with Three.js physically based materials. The materials use properties such as roughness, metalness, emissive colour, transparency and normal mapping. This allows the scene to react more convincingly to the different light sources in the museum and satisfies the project requirement of using lights together with multiple kinds of texture information.

Texture group	**Use in the project**
Floor textures	Wooden floor material. It uses colour, normal, and roughness maps to represent the visual appearance, surface relief, and reflection behaviour of the floor.
Wall textures	Concrete wall material. It uses colour, normal, and roughness maps to make the walls appear less flat and more realistic under the museum lighting.
Ceiling textures	Wooden ceiling panels. The diffuse image defines the visible wood pattern, while the displacement image is used as a bump map.
Painting images	Images used as textures for the paintings in the gallery. These textures can be changed interactively when the user clicks on a painting.
Procedural textures	Canvas-based textures generated in JavaScript and used for decorative elements such as wood effects, signs, and Egyptian-style surfaces.
Papyrus texture	Papyrus image used as the background of the HTML panel.

Table 6: Texture assets used in the project.

6 Lighting

The scene uses several types of Three.js lights summarized in Table 7.

The user can switch between the main and secondary lighting configurations. In the main lighting mode the room spotlights are active, the ambient contribution is stronger, the wall lamps are disabled, and the player's flashlight is turned off. In the secondary lighting mode the main spotlights are disabled, making the museum atmosphere darker, the wall lamps sources

Three.js light type	Use in the project
`THREE.AmbientLight`	General illumination of the museum scene.
`THREE.SpotLight`	Central atrium, dinosaur room, statue room, gallery, Egyptian room, dedicated T-Rex exhibit spotlight, left Moai face spotlight, right Moai face spotlight, and player flashlight attached to the right hand.
`THREE.PointLight`	Wall sconces distributed throughout the museum. Their light intensity is linked to the emissive intensity of the lamp material. These lights do not cast shadows.

Table 7: Three.js light types and their use in the museum scene.

are activated, and the player's flashlight is enabled, In this mode, the player raises the arm and aims the flashlight while moving through the museum.

7 Rendering

The renderer uses `THREE.SRGBColorSpace` for the final output. Colour textures, such as floor, wall, ceiling, procedural and painting maps, are also marked as sRGB, while normal, roughness and bump maps remain in linear data space because they store material data rather than colours.

Tone mapping is handled with `THREE.ACESFilmicToneMapping` and an exposure of `1.05`, giving smoother highlight compression. The shadow map uses percentage-closer filtering through `PCFSoftShadowMap`. The scene also uses exponential fog, whose colour and density change with the selected lighting mode.

The rendering-quality selector changes the maximum device pixel ratio and the shadow state:

- low quality uses a pixel ratio of 0.85 and disables shadows;
- medium quality uses a maximum pixel ratio of 1.25 and enables shadows;
- high quality uses a maximum pixel ratio of 1.75 and enables shadows.

8 Camera and views

To allow the user to experience the museum from different perspective modes, the application supports third-person, first-person, top-down and panoramic modes. All the views share the same `THREE.PerspectiveCamera`, with a field of view of 58 degrees, a near plane of 0.1 and a far plane of 160. When changing the view mode, the `CameraManager` updates the position and

orientation of the same camera object without creating a new camera.
In third-person mode, the camera follows the player from behind. Its yaw is updated when the player rotates, but it can also be adjusted by dragging the mouse. The pitch is clamped within a limited range to avoid extreme angles. To prevent walls from blocking the view, the system casts five rays from slightly different points around the player target toward the desired camera position. If an obstacle is detected, the camera distance is shortened while keeping a small safety margin from the obstacle.
In first-person mode, the camera is placed approximately 1.78 units above the player and slightly in front of the character. The camera yaw matches the player's rotation. The vertical mouse movement changes the pitch, used to control the aiming direction of the player's flashlight too.
In top-down mode, the camera is placed at position $(0, 52, 0)$ and looks down at the centre of the museum. Its up vector is changed so that the map orientation remains stable. In panoramic mode, the camera ignores the player's position. To create a cinematic effect, the camera slowly orbits around the museum at a radius of 31 unit. In both top-down and panoramic views, the ceilings and cornices are hidden so that the interior of the museum remains visible.
The mouse wheel controls the zoom of the perspective camera. The zoom value range is between 0.5 and 2.

9 Physics and Collision Handling

Since the project didnt require rigid-body simulation, forces, or realistic physical reactions, we didn't use a complete physics engine.
Instead, to prevent the player from crossing museum walls, exhibits, and the external boundaries of the scene, a custom collision system was implemented using `Three.js` bounding boxes.
The scene objects are approximated with invisible rectangular collision areas while the character is approximated as a vertical cylinder with a radius of approximately 0.48 world units. Before the player is moved to a new position, the system checks whether that position would overlap with one of these areas.
Rather than bounding boxes, the camera uses raycasting to avoid passing through walls in third-person mode, while interaction raycasting detects clicked exhibits.

10 How to run

To run the code use the link in the README.md or follow the Listing 3 instructions.

Listing 3: How to run *Night at the Museum*.

```
npm install
npm run dev
npm run build
// For a local preview of the production build:
npm run preview
```

11 User Interaction

The museum can be explored directly by controlling the security guard. With the `W` and `S` keys the player moves forward and backward relative to the direction in which the character is facing and, while `A` and `D` rotate the character left and right. In third-person mode, the camera follows the player's rotation while preserving its relative orbital position.

The camera system provides four viewpoints: third-person, first-person, top-down, and panoramic. The current viewpoint can be changed with `C`. The mouse wheel controls zoom, while mouse dragging rotates the camera in the third-view. In the first view the mouse drag change only the pith. The top-down camera displays the complete museum from above, whereas the panoramic camera automatically orbits around the scene.

The scene also contains several contextual interactions. The museum doors can be opened by pressing `O` when the player is close and facing them. Contextual prompts inform the user when an exhibit or painting can be selected. The T-Rex, Moai statue, sarcophagus, and paintings can be selected using mouse raycasting;

- clicking the T-Rex starts its roar animation and a one-shot tail swing;
- clicking the statue activates its movement and mouth animation;
- clicking the sarcophagus opens its lid, reveals the mummy, and automatically closes it after a short interval
- clicking a painting replaces its current artwork with an image not currently displayed by another painting.

The nearest painting begins to sway with an intensity proportional to the player's distance. Pressing `T` changes the state from quiet to awakening. All museum doors open, the lighting state changes, the T-Rex walks toward the atrium before entering its continuous roar animation, and the Moai statue hops from its room into the central area.

The HTML panel reports the current camera and lighting mode. It contains controls for main or secondary lighting, shadows, and low, medium, or high rendering quality. In secondary-light mode it reveals three range inputs for secondary-light intensity, flashlight intensity, and flashlight range.
The HTML panel can be shown or hidden by pressing the H key.

12 Conclusion

Night at the Museum is a browser-based 3D interactive scene that combines real time rendering, user interaction, lighting, textures and procedural animation. Three.js provides the WebGL rendering foundation, while the museum structure, gameplay logic, camera system and animations are implemented in JavaScript.
The project satisfies the main requirements through hierarchical animated models, several light types, different texture maps, interactive exhibits, configurable rendering options and four camera modes. All animations are created procedurally in code, without using imported animation clips.